Session 27 Homework — Notifications, Comment Deletion, and the Dashboard

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context (required)

From LIVE, you already have real comment create/list and a real inline `@mention` autocomplete. After completing the TEAMWORK task, you also have a real per-project Activity feed. This homework completes the remaining product flow on top of those pieces: a mentioned user has no real way to SEE their own notifications, nobody can delete a comment they posted by mistake, and there is still no single real place a signed-in user can check everything assigned to them across every workspace at once. This homework closes those gaps (CORE), then transfers the same pattern to a second real page (STRETCH), then asks you to reason in writing about the `@mention` token's own design (CHALLENGE).

Goal (required)

A real Notifications page listing every notification for the signed-in user, with working mark-one and mark-all-read controls (CORE); a real way to delete a comment, with a confirm step first (CORE); a real link to the Notifications page from anywhere in the app (CORE); a real cross-workspace Dashboard page showing assigned-to-me issues and recent activity (STRETCH), with a real link to it (STRETCH); a real shared error screen that catches any uncaught failure anywhere in the app instead of a raw crash page (STRETCH); a real, independent written answer explaining why a `@mention` token stores both the display name and the id (CHALLENGE).

What you already have

A real `GET /notifications/me?unread=`, `PATCH /notifications/:id/read`, and `PATCH /notifications/read-all` already exist on the backend — confirm their real shape yourself (don't guess it). `DELETE /comments/:id` already exists and works (author or workspace admin/owner only). `GET /users/me/issues` and `GET /users/me/activity` already exist and work.

What to build (required)

A real `AppNotification` frontend type; `markNotificationReadAction` / `markAllNotificationsReadAction`, both fire-and-forget Server Actions; a real Notifications page and a real `NotificationsList` Client Component with per-row mark-one and a page-level mark-all button; `deleteCommentAction` and a real, confirmed `DeleteCommentButton` wired into the existing comment list; a real link to `/notifications` from the app's own persistent account bar; a real Dashboard page fetching and rendering both of the already-existing `/users/me/*` routes; a real link to `/dashboard` from the same account bar; a real shared `error.tsx` boundary at the app's own root.

CORE (required)

How to start

Build the real notification-reading actions first (`markNotificationReadAction` / `markAllNotificationsReadAction`) — think about how they will actually be called (a button's `onClick`, not a `<form>` submission) before deciding whether they need to return anything at all. Then build the page and its own list component around data you can already fetch.

Hints

- **Hint 1 (conceptual):** think about what actually makes the Notifications page's own UI refresh after a mark-read click, if the action itself is never `await`ed by the caller.
- **Hint 2 (where to look):** `DeleteIssueButton` (Session 26) is the closest already-built real example of a button that calls a Server Action directly from `onClick`, with a confirm step first — read it before writing a new pattern from scratch.
- **Hint 3 (API or tool name):** `revalidatePath`, called from INSIDE the Server Action itself, not from the component that called it.
- **Hint 4 (product flow):** a notification's own `readAt` field tells you whether it has already been read — use it to decide whether the real "Mark read" button should render at all for a given row.
- **Hint 5 (comment delete):** `DeleteCommentButton` needs to know which comment to delete and which path to revalidate afterward — think about what props it actually needs, and where it will be rendered (inside the existing comment list, once per comment).

What to research

1. Why does a Server Action called directly from `onClick` (never through `useActionState`) need to THROW on failure instead of returning `{ error }`?
2. Why does `DeleteCommentButton` render for every real comment, with no check for whether the signed-in user is actually allowed to delete THIS one?

STRETCH (required)

How to start

Look at how `IssueEditPanel`'s own call site or the Dashboard's own description in "What to build" names its two real data needs (assigned issues; recent activity) — these come from two real, independent backend routes, not one combined one.

Hints

- **Hint 1 (conceptual):** think about why a Dashboard page needs its own local frontend types, rather than reusing `Issue` / `ActivityEntry` unchanged — what extra, nested data does each `/users/me/*` route actually return that those shared types don't already describe?
- **Hint 2 (where to look):** `ActivityFeed` (TEAMWORK) already has a real `describe()`-shaped lookup from a raw `action` value to a plain-English sentence — the Dashboard's own activity section needs something similar, adapted for its own richer, cross-workspace data.

- **Hint 3 (shared error boundary):** several of this session's own real call sites (`DeleteCommentButton`; `NotificationsList`'s fire-and-forget `markRead` / `markAllRead`) deliberately never wrap their own `deleteCommentAction` / `markNotificationReadAction` call in a local `try` / `catch`. Think about what currently happens, today, if one of those calls actually throws — and what Next.js's own file-based convention offers for catching an uncaught throw at the whole app's own root, the same way `not-found.tsx` already catches a missing route.

CHALLENGE (required)

How to start

Reread `MentionText`'s own real rendering logic and `extractMentionedUserIds`'s own real backend regex (or the backend reference doc), then write down, in your own words, what real problem each of the two stored pieces (the id, the name) actually solves on its own.

Hints

- **Hint 1 (conceptual):** think about what `MentionText` would need to do differently if a stored mention token held only an id, with no name at all.
- **Hint 2 (where to look):** think about what could go wrong if two real different users in the same workspace happened to share the exact same display name.

What to research

1. Why would storing only the id force `MentionText` into a real second network round-trip every time a comment is shown?
2. Why is a display name alone not a stable enough reference for the backend's own `extractMentionedUserIds` to rely on?

Rules (required)

Allowed: everything already shown LIVE/TEAMWORK this session (`CommentForm` / `CommentList`, `MentionTextarea` / `MentionText`, `ActivityFeed`, the generic list-with-per-row-action shape, `useActionState`, `apiFetch`), plus `DeleteIssueButton`'s own established pattern from Session 26.

Not yet: no keyboard navigation in the `@mention` dropdown, no debounced mention search, no navbar notification-count badge — none of that is part of this session's own scope.

Test it (required)

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
npm run build
```

Then, in a real browser: post a comment with a real `@mention` on one user's own issue, sign in as the mentioned user, open `/notifications`, confirm the real new notification appears unread, mark it read and confirm it is no longer offered a "Mark read" button, then use "Mark all read" on any remaining unread ones; sign in as the comment's own author and delete that comment, confirming a real confirm dialog appears first and the comment is actually gone after a reload; open `/dashboard` and confirm it lists issues actually assigned to the signed-in user and recent activity across every workspace they belong to; confirm both the Notifications and Dashboard pages are reachable from the account bar on any protected page; temporarily force a real uncaught error (e.g. point `DeleteCommentButton` at a bogus comment id) and confirm the real shared error screen renders instead of a raw crash page, then revert the temporary change.

Common confusions

- `await`-ing `markNotificationReadAction` / `markAllNotificationsReadAction` inside `onClick` and blocking the UI on it, instead of trusting the action's own `revalidatePath` to refresh the page once it finishes.
- Giving `DeleteCommentButton` a client-side check for whether the signed-in user "should" see it, instead of rendering it unconditionally and letting the backend enforce the real rule.
- Building one combined Dashboard fetch instead of the two real, independent calls (`GET /users/me/issues`, `GET /users/me/activity`) this backend actually offers.
- Answering the CHALLENGE with only one of the two required reasons (only "ids are stable" or only "names avoid a round-trip"), instead of connecting both to the same underlying idea.

Done when (required)

- `/notifications` lists every real notification for the signed-in user; mark-one and mark-all both actually work and persist after a reload.
- A comment can actually be deleted, with a real confirm step first, by its own author or a workspace admin/owner.
- `/notifications` is reachable from the app's own account bar.
- `/dashboard` lists real assigned-to-me issues and real recent activity across every workspace the signed-in user belongs to; it is reachable from the account bar.
- A real uncaught error anywhere in the app renders the real shared error screen, never a raw Next.js crash page, and "Try again" actually recovers.

- Your written CHALLENGE answer addresses BOTH real reasons a `@mention` token stores the id AND the name, not just one of them.
- `npx tsc --noEmit` and `npm run build` both exit clean.

Working time (required)

About 94 minutes total (HW-01…22) — CORE (HW-01…15) is ~58 minutes; STRETCH (HW-16…20, HW-22) is ~26 minutes; CHALLENGE (HW-21) is ~10 minutes.